

Architecture — Xolvon Backend

Code-verified architecture reference for the NestJS 12 (strict ESM, Express platform) backend. Every claim below cites the file that implements it.

Companion documents: `api-contract.md` (endpoint contract), `security.md` (controls), `deployment-runbook.md` (operations), `decision-log.md` (why).

1. Request lifecycle

Bootstrap order in `src/main.ts` (all guards/pipes below are process-wide):

```
Request → Express (trust proxy: TRUST_PROXY_HOPS, src/main.ts:25–28)
  → Helmet security headers (enableSecurityHeaders, src/main.ts:35–39)
  → CORS (explicit origin whitelist + credentials, src/main.ts:43–50)
  → Global prefix /api (src/main.ts:17)
  → APP_GUARD #1 ThrottlerGuard (src/app.module.ts:65)
  → APP_GUARD #2 RequestIntegrityGuard (src/app.module.ts:66)
  → @UseGuards(AuthGuard, RolesGuard) where declared per route/controller
  → ValidationPipe { whitelist, forbidNonWhitelisted, transform } (src/main.ts:53–59)
  → Controller → Service → DatabaseService token / StoragePort token
  → AllExceptionsFilter (safe JSON error envelope, src/main.ts:62)
```

There is no middleware layer and no `NestModule` configuration anywhere — the only `app.use()` is Helmet inside `src/security/security-headers.ts`.

`AuthGuard` is deliberately **not** global: routes that don't attach it are public (catalog/search/home/collective/marketplace GETs); `@Public()` is metadata consumed by `AuthGuard` only where it is attached (`src/auth/guards/auth.guard.ts:23–30`).

2. NestJS module map

`src/app.module.ts:30–61` (import order):

Module	Directory	Responsibility
<code>ConfigModule</code> (global)	<code>src/config/</code>	<code>.env</code> load + fail-fast <code>validateEnvironment</code> (<code>env.validation.ts</code>); CORS, CSP, trust-proxy, throttle resolvers
<code>ThrottlerModule</code> (global, via <code>APP_GUARD</code>)	<code>src/config/throttle.config.ts</code>	5 named limit groups (§5.3)
<code>DatabaseModule</code>	<code>src/database/</code>	<code>DatabaseService</code> DI token; driver factory <code>sqlite</code> ↔ D1 REST adapter; migration runner
<code>AuthModule</code>	<code>src/auth/</code>	Register/login/refresh/logout, Argon2id hashing, JWT issue/verify, hashed refresh sessions, <code>AuthGuard</code> / <code>RolesGuard</code>
<code>EnrollmentsModule</code>	<code>src/enrollments/</code>	Entitlement checks, idempotent activation, admin revoke
<code>OrdersModule</code>	<code>src/orders/</code>	Server-computed order totals, payment-proof submission (anti-IDOR), verify/activate/cancel transitions
<code>CoursesModule</code>	<code>src/courses/</code>	Published-only catalog, draft-gated admin CRUD, publish gates, slug uniqueness
<code>HomeModule</code>	<code>src/home/</code>	Published-only aggregation (featured courses/projects/marketplace/collective), no N+1

Module	Directory	Responsibility		
MarketplaceModule	src/marketplace/	Showcase-only SaaS catalog; <code>externalUrl</code> minimal https validation (DL-013)		
AuditModule	src/audit/	Internal <code>record()</code> writer for <code>admin_audit_logs</code> ; no public endpoint, metadata never exits the API		
CollectiveModule	src/collective/	Talent roster (draft/published), never exposes email/phone on public reads		
ProjectsModule + ProjectMediaModule + ProjectMembersModule	src/projects/ , src/project-media/ , src/project-members/	Portfolio CRUD, media attach/detach (private object keys never public), member assignment (<code>type</code> / <code>mediaType</code> / <code>role</code> are controlled strings, DL-014)		
LessonsModule	src/lessons/	Lesson CRUD/reorder/publish; <code>GET /api/lessons/:id/video-url</code> mints 300s signed URLs gated by active enrollment		
CourseResourcesModule	src/course-resources/	Resource attach ('pdf'	resource\	assignment') under lessons
AdminModule	src/admin/	Read-only admin surfaces: users, orders, overview counters		
MediaModule	src/media/	<code>STORAGE_PORT</code> binding + admin upload-url/confirm/read-url endpoints		
ProgressModule	src/progress/	Lesson completion state, ownership from JWT subject		
SearchModule	src/search/	Cross-entity FTS5 search (courses/projects/marketplace), published-only		

Health endpoint: `GET /api (src/app.controller.ts)` — returns a static

string, exempt from every throttle group via

`@SkipThrottle(HEALTH_SKIP_THROTTLE)`.

3. Data layer — one token, two drivers

All services inject the `DatabaseService` class as DI token; the factory in

`src/database/database.module.ts:25–44` selects the implementation from

`DB_DRIVER`:

3.1 sqlite (local development & e2e)

`DatabaseService (src/database/database.service.ts)` opens `better-sqlite3`

on `./local.db` (relative to cwd) with `foreign_keys = ON`, exposing

`queryAll`, `queryOne`, `execute` — always positional ? bind parameters

(prepared statements are mandatory per HANDBOOK_BACKEND §4.1). Zero cloud

credentials needed.

3.2 d1 (staging/production)

`D1RestAdapter (src/database/d1-rest.adapter.ts)` wraps `D1Service`

(`src/database/d1.service.ts`) behind the same three methods. `D1Service`

POSTs `{sql, params}` to

`https://api.cloudflare.com/client/v4/accounts/<CLOUDFLARE_ACCOUNT_ID>/d1/database/<CLOUDFLARE_D1_DATABASE_ID>/query`

with `Authorization: Bearer <CLOUDFLARE_API_TOKEN>` **resolved on demand per

query and never stored as an instance field** (hardening invariant,

`d1.service.ts:77–79`). Timeout 10s; timeouts→504, network/non-OK→502 — the

adapter never leaks the upstream body. `onModuleInit` fail-fasts on missing

`CLOUDFLARE_*` values. (`src/database/d1.module.ts` exists but is imported

nowhere — dead code kept for reference.)

✗ Migrations run through the same `execute()` contract, so the runner

(`src/database/migrate.ts`) works on either driver; the operational path for

D1 is `wrangler d1 migrations apply` (see runbook). Applied files are recorded

in `_migrations` with a SHA-256 hash; hash drift throws (fail-closed,

forward-only policy).

4. Storage layer — `StoragePort` interface-first (DL-007)

`src/media/storage.port.ts` defines the token `STORAGE_PORT` and the interface:

```
createUploadUrl(request: { key; contentType; contentLength }): Promise<string>
createReadUrl(key: string, expiresIn?: number): Promise<string>
confirmUpload(key: string): Promise<void>
```

`src/media/media.module.ts:12` binds the implementation from

`STORAGE_DRIVER`:

Driver	Class	Behavior
local-test	<code>src/media/local-test-storage.service.ts</code>	Deterministic stub URLs (<code>http://local-storage.test/...</code>); <code>confirmUpload</code> records keys in memory. Used by all unit/e2e runs — tests never touch the network.
r2	<code>src/media/r2-storage.service.ts</code>	S3 SDK v3 against R2 (<code>region: 'auto'</code>); presigned PUT 3600s (signable headers: <code>content-type/content-length</code>) and presigned GET default 300s . Credentials only from env, never in responses.

Consumers (`src/media/media.service.ts` , `src/lessons/lessons.service.ts`)

inject the token and are unchanged when drivers are swapped. Private object

keys never appear in public API responses; entitlement is checked **before**

a read URL is minted (server owns trust).

5. Auth model — JWT Bearer + hashed refresh sessions

5.1 Tokens

- **Access:** JWT signed/verified with `JWT_SECRET`, TTL **15m**

(`src/auth/auth.service.ts:132–133`, `211–212`). Sent as

`Authorization: Bearer <token>`; extraction and verification in

`src/auth/guards/auth.guard.ts` (generic 401 messages, no token echo).

- **Refresh:** cryptographically random string returned once at login; the

database stores only its **SHA-256 hash** in `sessions.refresh_token`

(`auth.service.ts:31–33`, `136–149`) alongside `user_id`, `ip_address`,

`user_agent`, `expires_at`. `refresh` looks the session up by hash and

purges expired rows; `logout` deletes the session by hash

(`auth.service.ts:173–225`). Plaintext refresh tokens can never be read back

from a leaked database.

5.2 CSRF model (explicit non-decision to adopt cookies)

The cookie-session CSRF pattern (synchronizer token in a cookie) was ****NOT**

adopted**. The API is Bearer-only — no authentication happens via cookies —

so classic cross-site request forgery cannot carry credentials

automatically. Instead, browser state-changing requests are constrained by the

request-integrity guard: `POST` / `PATCH` / `DELETE` requests must not declare

`Sec-Fetch-Site: cross-site`, and any `Origin / Referer` they carry must be in

the explicit allowed origin set (`CORS_ALLOWED_ORIGINS`, falling back to

`FRONTEND_URL`). Non-mutating requests are unaffected, and headerless

service-to-service clients remain compatible; authentication is still

required independently on every protected route. This is the locked decision

DL-018 (with DL-003 keeping the repo's JWT+refresh model over the

handbook's `getSession()` sample); implementation

`src/common/guards/request-integrity.guard.ts`, rationale

`security.md`.

5.3 Rate limiting

`@nestjs/throttler` with five named groups, all 60s windows

(`src/config/throttle.config.ts`): `default` 100/min (undecorated routes),

`search` 30/min (`GET /api/search`), `catalog` 30/min (`courses/projects/`

`marketplace/collective` listings + `GET /api/home`), `signed` 10/min (lesson

video URL, `media upload-url/read-url`), `order` 10/min (`POST /api/orders`).

`POST /api/auth/login` and `POST /api/auth/register` are tightened inline to

5/min per IP (DL-012/DL-016). Client-IP accuracy behind an edge proxy is

controlled by `TRUST_PROXY_HOPS`; locally it fails **closed** to `false` so

`X-Forwarded-For` cannot mint fresh buckets (`src/config/trust-proxy.ts`).

5.4 Authorization

`RolesGuard` enforces `@Roles('admin')` on all admin surfaces; roles come

exclusively from the `users` row / verified JWT — never from request input

(DL-005). First-admin creation is a controlled seed command

(`npm run seed:admin`), not a public route.

6. Data model — 19 tables + FTS5

Schema source of truth: `SCHEMA.md`; applied set:

`src/database/migrations/0000...0007` (19 regular tables + 3 FTS5 virtual

tables; plus the runtime bookkeeping table `_migrations`). Conventions

(HANDBOOK_BACKEND §3.1): TEXT UUID v4 primary keys, ISO 8601 TEXT timestamps,

INTEGER money (Rupiah) and booleans (0/1), TEXT + `CHECK` for enums,

prepared statements only.

6.1 Tables by domain

Domain	Tables	Notes		
Identity	<code>users</code> , <code>sessions</code>	<code>users.role</code> CHECK 'user'	<code>admin</code> ; <code>users.status</code> defaults 'active' (no CHECK constraint); <code>sessions.refresh_token</code> stores SHA-256 hash (§5.1)	
Course	<code>courses</code> , <code>lessons</code> , <code>course_resources</code> , <code>course_tags</code>	<code>courses.slug</code> UNIQUE; <code>lessons.order_index</code> sequencing, <code>is_preview</code> 0/1, <code>video_object_key</code> private; <code>resource type</code> 'pdf'	<code>resource\</code>	<code>assignment'</code>

Domain	Tables	Notes		
Xolvon Backend Transaction	Documentation · architecture.md orders , order_items , payment_proofs	orders.status final 'pending'	paid\	cancelled (DL-002); server-computed orders.amount (CHECK > 0); proofs store R2 object keys + uploaded_by', anti-IDOR at submit (DL-006)
Entitlement	enrollments	UNIQUE (user_id, course_id) prevents double enrollment; status final 'active'	revoked' (DL-004); idempotent upsert on activation	
Learning	progress	Per-user lesson completion; ownership from JWT subject		
Portfolio	projects , project_media , project_tags	Narrative fields Problem→Solution→Tech→Result; project_media.object_key private, media_type controlled string (DL-014)		
Collective	collective_members , project_members	project_members(project_id, member_id) is the Portfolio→Collective M:N bridge with explicit role; display_order for listings		
Marketplace	marketplace_items , marketplace_media	Showcase only — no transactions (SCHEMA.md §60); external_url https-minimal (DL-013)		
Security	admin_audit_logs	Append-only trail for admin state mutations (verify/activate/cancel/revoke/publish/create/update/delete); never exposed publicly		
Search	course_fts , project_fts , marketplace_fts	FTS5 virtual tables kept in sync by 9 insert/delete/update triggers (0007_fts.sql); power GET /api/search		

6.2 Relationships

```

users 1-N sessions          (user_id FK)
users 1-N orders             (user_id FK)
users 1-N enrollments  N-1 courses (both FKs; UNIQUE pair)
users 1-N progress          (user_id FK; lesson-scoped rows)
courses 1-N lessons         (course_id FK, ON DELETE CASCADE)
courses 1-N course_resources (via lesson/course, course_resources FK)
courses N-M course_tags
orders 1-N order_items  N-1 courses (order_id, course_id FKs)
orders 1-N payment_proofs (order_id FK; object keys only)
projects 1-N project_media (project_id FK)
projects N-M collective_members (via project_members: composite PK
                                (project_id-projects FK CASCADE, member_id – lookup enforced in code)
projects N-M project_tags
marketplace_items 1-N marketplace_media
users 1-N admin_audit_logs   (actor_user_id FK)

```

Critical invariants (HANDBOOK §3.3, DL-002/DL-004): entitlement is

enrollments.status='active' per (user, course); expires_at columns are

enforced in queries; no delete/role-change endpoints exist in V1, so

soft-state transitions (revoke/cancel/unpublish) are the only mutations, and

each writes an audit row.

7. Framework notes

- **ESM-only:** NestJS 12 with "type": "module"; hence

NODE_OPTIONS=--experimental-vm-modules in test scripts (DL-008) and

.js -extension imports in source.

- **Deviation from HANDBOOK_BACKEND §8** (Langkah 6/8): the handbook's

Xolv3r Next.js Workers binding path targets the frontend/Workers deployment. The backend is Node/Express and reaches D1 through the REST adapter behind the `DatabaseService` token — a documented deviation, conflict K-06 in `decision-log.md`.

- **Error surface:** `AllExceptionsFilter` returns

`{statusCode, message, error, timestamp, path}` with a generic 500 message — no stack traces, upstream bodies, or secrets.